

Step 7: Cross-Validation (LOOCV)

Danish Housing Prices · Mathematical Statistics

Carlos Castillo

2026-05-14

```
library(tidyverse)
dk_villa <- readRDS("../data/dk_villa.rds")
M_final  <- readRDS("../data/M_final.rds")

cat("Model formula:", deparse(formula(M_final)), "\n")
cat("n =", nrow(dk_villa), "\n")
```

```
Model formula: log_price ~ log_sqm + house_age + nom_interest_rate. + dk_ann_infl_rate.
n = 402
```

1 Motivation

The training error $s(M)$ measures how well the model fits the data it was trained on. If the model is overfitted — i.e. too many predictors selected to match noise; $s(M)$ will be optimistically small and predictions on new data will be poor.

Leave-one-out cross-validation (LOOCV) estimates prediction error on genuinely unseen data without requiring a separate holdout set.

2 The LOOCV Formula

Naïvely, LOOCV requires refitting the model n times. But we can follow a shortcut via the **hat matrix**:

$$\tilde{e}_i = \frac{e_i}{1 - h_{ii}}$$

where $e_i = Y_i - \hat{Y}_i$ is the ordinary residual and h_{ii} is the i -th diagonal of the hat matrix $H = X(X^\top X)^{-1}X^\top$.

The LOOCV estimate of σ is then:

$$s_{cv} = \sqrt{\frac{1}{n} \sum_{i=1}^n \tilde{e}_i^2}$$

Intuition for the shortcut: h_{ii} measures how much observation i “pulls” the fitted line toward itself. Dividing by $(1 - h_{ii})$ inflates the residual to approximate what it would have been if i had been left out during fitting. High-leverage points (h_{ii} close to 1) get inflated most.

3 Computation

```
h      <- hatvalues(M_final)
e      <- residuals(M_final)
e_loo <- e / (1 - h)

s_M    <- summary(M_final)$sigma
s_cv   <- sqrt(mean(e_loo^2))

cat("Training error:  s(M)  =", round(s_M, 5), "\n")
cat("LOOCV error:     s_cv  =", round(s_cv, 5), "\n")
cat("Ratio s_cv/s(M)   =", round(s_cv / s_M, 4), "\n\n")

cat("Rule of thumb:\n")
cat("  ratio ~ 1.0 : model generalises well\n")
cat("  ratio >> 1.0: model is overfitting\n")
```

```
Training error:  s(M)  = 0.27463
LOOCV error:     s_cv  = 0.27656
Ratio s_cv/s(M)   = 1.007
```

```
Rule of thumb:
  ratio ~ 1.0 : model generalises well
  ratio >> 1.0: model is overfitting
```

4 Leverage Diagnostics

High-leverage observations ($h_{ii} > 2p/n$ where p is the number of parameters) are worth examining — they disproportionately influence the fit.

```
p <- length(coef(M_final))
n <- nrow(dk_villa)
thr <- 2 * p / n

cat("Leverage threshold 2p/n =", round(thr, 4), "\n")
cat("Observations above threshold:", sum(h > thr), "\n\n")

# Top 10 highest-leverage points
top_lev <- order(h, decreasing = TRUE)[1:10]
cat("Top 10 high-leverage observations:\n")
print(data.frame(
  obs      = top_lev,
  leverage = round(h[top_lev], 4),
  loo_resid = round(e_loo[top_lev], 4),
  log_price = round(dk_villa$log_price[top_lev], 4)
))
```

Leverage threshold $2p/n = 0.0249$

Observations above threshold: 24

Top 10 high-leverage observations:

	obs	leverage	loo_resid	log_price
340	340	0.1083	-0.0199	15.8495
110	110	0.0723	-0.8772	14.8905
105	105	0.0654	-0.4686	15.2971
254	254	0.0525	-0.0166	15.4709
143	143	0.0513	0.1803	15.5947
98	98	0.0478	-0.1023	14.8576
379	379	0.0462	-0.0138	15.3751
96	96	0.0421	0.1225	15.8170
235	235	0.0359	0.1120	15.5472
253	253	0.0357	-0.0449	15.5734

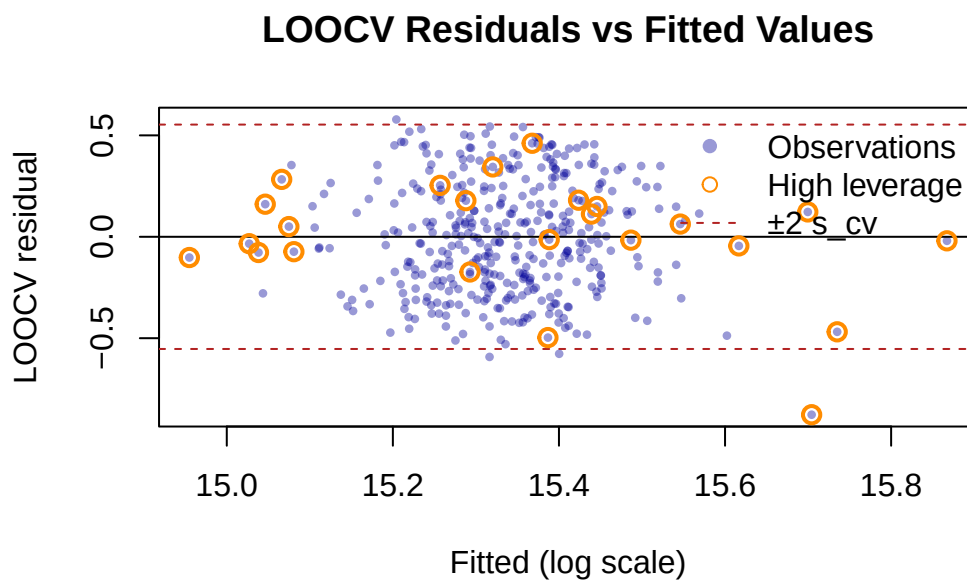
5 LOOCV Residual Plot

```

plot(fitted(M_final), e_loo,
     pch = 16, cex = 0.6, col = rgb(0, 0, 0.6, 0.4),
     main = "LOOCV Residuals vs Fitted Values",
     xlab = "Fitted (log scale)", ylab = "LOOCV residual")
abline(h = 0, lty = 1)
abline(h = 2*s_cv, lty = 2, col = "firebrick")
abline(h = -2*s_cv, lty = 2, col = "firebrick")

# Highlight high-leverage points
points(fitted(M_final)[h > thr], e_loo[h > thr],
       pch = 1, cex = 1.2, col = "darkorange", lwd = 2)
legend("topright",
      legend = c("Observations", "High leverage", " $\pm 2 s_{cv}$ "),
      pch = c(16, 1, NA),
      lty = c(NA, NA, 2),
      col = c(rgb(0,0,0.6,0.4), "darkorange", "firebrick"),
      bty = "n")

```

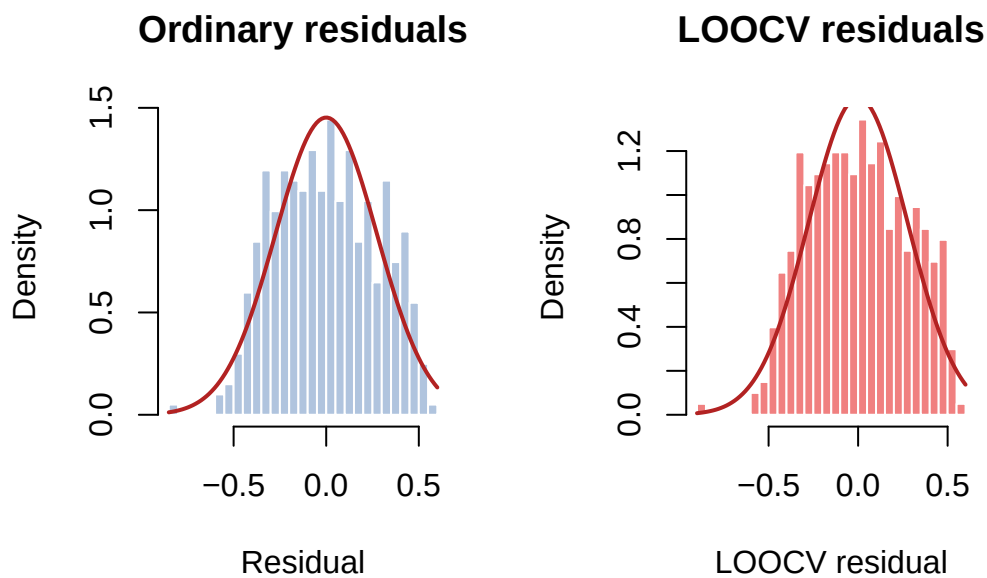


6 Comparison: Training Error vs LOOCV Error

```
# Distribution of ordinary vs LOOCV residuals
par(mfrow = c(1, 2))

hist(e, breaks = 30, prob = TRUE,
     col = "lightsteelblue", border = "white",
     main = "Ordinary residuals",
     xlab = "Residual")
curve(dnorm(x, 0, s_M), add = TRUE, col = "firebrick", lwd = 2)

hist(e_loo, breaks = 30, prob = TRUE,
     col = "lightcoral", border = "white",
     main = "LOOCV residuals",
     xlab = "LOOCV residual")
curve(dnorm(x, 0, s_cv), add = TRUE, col = "firebrick", lwd = 2)
```



```
par(mfrow = c(1, 1))
```

7 Summary Table

```
res_tbl <- data.frame(
  Metric = c("s(M) - training error",
             "s_cv - LOOCV error",
             "Ratio s_cv / s(M)",
             "Number of parameters p",
             "n"),
  Value = c(round(s_M, 5),
             round(s_cv, 5),
             round(s_cv / s_M, 4),
             p,
             n)
)
knitr::kable(res_tbl, caption = "LOOCV summary")
```

Table 1: LOOCV summary

Metric	Value
s(M) — training error	0.27463
s_cv — LOOCV error	0.27656
Ratio s_cv / s(M)	1.00700
Number of parameters p	5.00000
n	402.00000

Interpretation on DKK scale: Since the model is on the log scale, multiply s_{cv} by the typical price to get an approximate absolute prediction error. For a 4.5M DKK villa: $4,500,000 \times (e^{s_{cv}} - 1) \approx$ prediction error in DKK.

```
median_price <- exp(mean(dk_villa$log_price))
approx_error <- median_price * (exp(s_cv) - 1)
cat("Approximate LOOCV prediction error (DKK):",
    format(round(approx_error), big.mark = ","), "\n")
```

Approximate LOOCV prediction error (DKK): 1,455,394